AliShahali1382's blog

Codeforces Round #722 Editorial

By AliShahali1382, history, 5 years ago,

Hi, Codeforces!

1529A - Eshag Loves Big Arrays

problem idea and solution: **AmShZ**
- editorial
- official solution

1529B - Sifid and Strange Subsequences

problem idea and solution: **Davoth**
- editorial
- official solution

1528A - Parsa's Humongous Tree

problem idea and solution: **shokouf**, **AmShZ**
- editorial

1528A - Parsa's Humongous Tree

The solution is based on the fact that an optimal assignment for a exists such that for each vertex v, $a_v \in l_v, r_v$.

Proving this fact isn't hard, pick any assignment for a. Assume v is a vertex in this assignment such that $a_v \notin l_v, r_v$.

Let p be the number of vertices u adjacent to v such that $a_u > a_v$.

Let q be the number of vertices u adjacent to v such that $a_u < a_v$.

Consider the following cases:

- $p > q$: In this case we can decrease a_v to l_v and get a better result.
- $p < q$: In this case we can increase a_v to r_v and get a better result.
- $p = q$: In this case changing a_v to l_v or r_v will either increase or not change the beauty of the tree.

Based on this fact, we can use dynamic programming to find the answer.

Define $dp_{v,0}$ as the maximum beauty of v's subtree if a_v is equal to l_v.

Similarly, define $dp_{v,1}$ as the maximum beauty of v's subtree if a_v is equal to r_v.

$dp_{v,j}$ is calculated based on v's children, for each of v's children such as u, we add u's contribution to $dp_{v,j}$.

The transitions are:

- $dp_{v,0} += max(dp_{u,0} + |l_v - l_u|, dp_{u,1} + |l_v - r_u|)$
- $dp_{v,1} += max(dp_{u,0} + |r_v - l_u|, dp_{u,1} + |r_v - r_u|)$

It's clear that the answer is equal to $max(dp_{v,0}, dp_{v,1})$.

complexity: $\mathcal{O}(n)$

- official solution

```
// Call my Name and Save me from The Dark
#include <bits/stdc++.h>

using namespace std;

typedef long long int ll;
typedef pair<int, int> pii;

#define SZ(x)                       (int) x.size()
#define F                           first
#define S                           second

const int N = 2e5 + 10;
ll dp[2][N]; int A[2][N], n; vector<int> adj[N];

void DFS(int v, int p = -1) {
    dp[0][v] = dp[1][v] = 0;
    for (int u : adj[v]) {
        if (u == p) continue;
        DFS(u, v);
        dp[0][v] += max(abs(A[0][v] - A[1][u]) + dp[1][u],
dp[0][u] + abs(A[0][v] - A[0][u]));
        dp[1][v] += max(abs(A[1][v] - A[1][u]) + dp[1][u],
dp[0][u] + abs(A[1][v] - A[0][u]));
    }
}

void Solve() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%d%d", &A[0][i], &A[1]
[i]);
    fill(adj + 1, adj + n + 1, vector<int>());
    for (int i = 1; i < n; i++) {
        int u, v; scanf("%d%d", &u, &v);
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    DFS(1);
    printf("%lld\n", max(dp[0][1], dp[1][1]));
}

int main() {
    int t; scanf("%d", &t);
    while (t--) Solve();
    return 0;
}
```

1528B - Kavi on Pairing Duty

problem idea and solution: **alireza_kaviani**
- editorial
- official solution

1528C - Trees of Tranquillity

problem idea and solution: **AliShahali1382**
- editorial
- official solution
- better implementation of official solution: AaParsa
- python solution: Tet
- another O(n.log) solution with seg-tree: N.N_2004

1528D - It's a bird! No, it's a plane! No, it's AaParsa!

problem idea and solution: **AmShZ**, **AliShahali1382**
- editorial
- official solution
- a different O(n^3) solution: Atreus

1528E - Mashtali and Hagh Trees

problem idea and solution: **AliShahali1382**

Thanks to **Kininarimasu** for the editorial.
- editorial
- official solution

1528F - AmShZ Farm

problem idea and solution: **AmShZ**, **AliShahali1382**
- editorial
- official solution

Tutorial of Codeforces Round 722 (Div. 1)
Tutorial of Codeforces Round 722 (Div. 2)
cf round 722, #editorial

+260 | AliShahali1382 | 5 years ago | 191

Comments (153)

☐ Show archived | Write comment?

AliShahali1382 5 years ago, hide # | ☆ +38
Auto comment: topic has been updated by AliShahali1382 (previous revision, new revision, compare).
→ Reply

E404_Not_Found 5 years ago, hide # | ☆ +31
Woow !! Fast editorial Thanks :)
→ Reply

tdpencil 5 years ago, hide # | ☆ ← Rev. 3 +32
There might be an error in the n log n solution with segtree (div1. problem C).
→ Reply

AliShahali1382 5 years ago, hide # ^ | ☆ +21
fixed, thanks!
→ Reply

N.N_2004 5 years ago, hide # | ☆ +196
I hope all the future rounds will be as good as this one
→ Reply

Sumarbrander 5 years ago, hide # ^ | ☆ +44
I hope all future rounds

To be continued... :(
→ Reply

TheNoOneCoder 5 years ago, hide # | ☆ +49
Nice contest with nice problems! Thanks
→ Reply

AmShZ 5 years ago, hide # | ☆ +221
Genius problems. Nice job.
→ Reply

Tet 5 years ago, hide # ^ | ☆ +433

→ Reply

mango_lassi 5 years ago, hide # | ☆ ← Rev. 2 +26
Bad tests in D? The $\log n$ part in my $\mathcal{O}(nm \log n)$ code triggers only $0.025 N^3$ times in large tests: 117257732

EDIT: **dorijanlendvaj** found a large input where it triggers $0.5 N^3$ times, that's 20 times better than the original test data!
→ Reply

AliShahali1382 5 years ago, hide # ^ | ☆ +14
we wanted to fail high-constant O(n^3.log)(like seg-trees) and let easily optimized O(n^3) or O(n^3.log) with heaps to pass.
→ Reply

-senshi- 5 years ago, hide # | ☆ ← Rev. 2 +148

Sifid has an array a of n integers. Sifid likes everything big,

Via 1529B — Sifid and Strange Subsequences
→ Reply

Davoth 5 years ago, hide # ^ | ☆ +51
This is what happens when you don't help in making statements. :(
→ Reply

Sachit_Jaggi 5 years ago, hide # | ☆ ← Rev. 3 0
Dfs Based Solution of C Parsa's Humongous Tree without DP based on the fact that during path traversal on a tree from root , A particular node is reached only once.

We return a pair {a,b} from every node.

A has the best solution from that node to all nodes below it when left_mini is assigned to that node.

B has the best solution from that node to all nodes below it when right_maxi is assigned to that node.

We store best possible values of the current node from all children node i.e. one with maxi on all possible calls and other with mini and then return it.

In the end when recursive calls end print the maximum of the pair of values returned from the root.

Solution
→ Reply

pandastic 5 years ago, hide # ^ | ☆ 0
What you are describing is the same DP on a tree. The official solution also uses DFS.
→ Reply

Sachit_Jaggi 5 years ago, hide # ^ | ☆ +3
U don't need to save results in a array for Dp in this DFS

Both best case possibilities are calculated and return from there.
→ Reply

SirShokoladina 5 years ago, hide # | ☆ +38
Where is the editorial on F?..
→ Reply

Tet 5 years ago, hide # ^ | ☆ +14
I've added it but it doesn't seem to appear ...

will probably get fixed soon
→ Reply

valiente 5 years ago, hide # | ☆ ← Rev. 3 +2
For Div2 D- "x≤n: In this case, due to the lemma mentioned above all the segments must have the same length, thus their length must be a divisor of n, in this case they can be paired in D(n) ways; where D(n) is the number of divisors of n.".

The equality should not be present I think. It should be only x<n. Correct me if I am wrong!
→ Reply

Tet 5 years ago, hide # ^ | ☆ +10
If 1 is paired with n, each number from n+1 to 2n must belong in a segmemt with length n, where the length of a segment is defined as the number of elements inside it.

The pairing is unique as well ...
→ Reply

dush1729 5 years ago, hide # | ☆ +106
- Spoiler

→ Reply

traxex2 5 years ago, hide # ^ | ☆ 0
The sequence x[i]-2*x[i-1] starts 1,0,1,-1,2,... and is http://oeis.org/A051950
→ Reply

Venom_ous 5 years ago, hide # | ☆ 0
I have done Question b in two way 1st submission : 117264950 2nd submission : 117264953 I can't find why my second submission failed I did the same thing but just in another manner, can anyone help me out please
→ Reply

7_7 5 years ago, hide # ^ | ☆ +5
It took me a lot of time but I found out what's wrong with your code. Actually, you are failing edge case when seq is [-1] or [0] because in your second program you'll be having cnt = 1 and since there is only 1 item in the array it will not enter the while loop, and bool will evaluate to true and it will increase it by 1 resulting in ans = 2. Luckily your former code worked for these edge cases.
→ Reply

Venom_ous 5 years ago, hide # ^ | ☆ 0
First thank you very much for your efforts Could you please help me this submission , it is failing on 10000th test case 117240464
→ Reply

7_7 5 years ago, hide # ^ | ☆ 0
I have submitted your code with a minor change, it got ac. You just had to account for the special case when n equals 1.
→ Reply

-3en_7oras- 5 years ago, hide # | ☆ ← Rev. 2 +23
i am sorry for every one get wrong answer in problem B test 15 i put it in hacks :(can you give contriubution for it
→ Reply

Darshan_Bhangale 5 years ago, hide # | ☆ +1
Can anyone explain the DP applied in Kavi on the Pairing Duty problem in simple words?
→ Reply

OleschY 5 years ago, hide # | ☆ +8
In **Div 1 Problem D**:

"To handle the "waiting" concept, we can add n fake edges, i-th of which is from the i-th vertex to the (i+1 mod n) -th vertex with weight equal to one."

What is a vertex here? Does this sentence mean the cities? But waiting for 1 second doesn't move you to the next city, so that can't be. Or can it? Is the assumption, that "if we can reach city X from some other city Y, then we could've waited for 1s in Y and then we would've reached X+1"? That seems also to be the explanation for "It can be proved that doing dijkstra in the new graph is sufficient if we guarantee that the first used edge is not fake."

Well, It seems I answered my own question here while writing it down. Debugging-Rubber-Ducky for the win. But since I really was confused by this and I don't feel that this step is trivial, I'm gonna leave this comment here.
→ Reply

-this-is-obd- 5 years ago, hide # ^ | ☆ +8
"It can be proved that doing dijkstra in the new graph is sufficient if we guarantee that the first used edge is not fake.

We can map waiting for x seconds and then using an edge to go to u from v to using a cannon and then using x fake edges to go to u from v."
→ Reply

OleschY 5 years ago, hide # ^ | ☆ 0
Oh, yes you are right. I'm just thick as a brick and didn't get that sentence before. Thanks for the heads up! :)
→ Reply

FlakeLCR 5 years ago, hide # | ☆ 0
Can someone explain how to find Stirling numbers of the 2nd kind in O(k log k) (for F)?
→ Reply

aksim1744 5 years ago, hide # ^ | ☆ ← Rev. 2 +126
From wikipedia page

$$S(n,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

You can rewrite it as

$$S(n,k) = \sum_{i=0}^{k}\underbrace{\left((-1)^i \cdot \frac{1}{i!}\right)}_{a_i} \cdot \underbrace{\left(\frac{1}{(k-i)!} \cdot (k-i)^n\right)}_{b_{k-i}}$$

So you can calculate arrays a and b, and stirling numbers are just convolution of these arrays:

$$S(n,k) = \sum_{i=0}^{k} a_i \cdot b_{k-i}$$

→ Reply

aakrosh 5 years ago, hide # | ☆ 0
I am getting TLE for Div 2 C Mysolution. I have done almost the same as Editorial (using memorization of recursion). Please anyone help me with debugging. Thank you
→ Reply

izlyforever 5 years ago, hide # ^ | ☆ ← Rev. 2 +10
`unordered_map` and `unordered_set` are dangerous in CP
→ Reply

aakrosh 5 years ago, hide # ^ | ☆ 0
Thank you for replying. I want to know What is the exact reason not to use unordered_map and unordered_set.
→ Reply

izlyforever 5 years ago, hide # ^ | ☆ 0
Here
→ Reply

-this-is-obd- 5 years ago, hide # ^ | ☆ 0
*in CF not CP
→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 2 0

bojackhorseman550

I am getting a TLE too on my Div2C. Can someone help me with debugging too ? Thank You. [My Solution]http://codeforces.com/contest/1529/submission/117308814)

→ Reply

_winprn

5 years ago, hide # ^ | ☆ 0

just add fastIO to your submission bc input quite large. Same code but has fastIO

→ Reply

Kininarimasu

5 years ago, hide # | ☆ ← Rev. 11 +134

Since E editorial is not out yet, here is how I solved. Let dp_i be the answer for all trees such that there exists a root and all edges are directed in the same direction from root and the root has at most 2 children. We transition

$dp_i = dp_{i-1} + dp_{i-1} * pdp_{i-2} + dp_{i-1} * (dp_{i-1} + 1)/2$ where $pdp_i = \sum_{j=0}^{i} dp_j$

Then let $dp2_i$ be the same as dp_i except the tree must have 2 children. So $dp2_i = dp_i - dp_{i-1}$.

The answer for these cases is

$$2 * (dp_n + dp_{n-1} * pdp_{n-2} * (pdp_{n-2} + 1)/2 + pdp_{n-2} * dp_{n-1} * (dp_{n-1} + 1)/2 + dp_{n-1} * (dp_{n-1} + 1) * (dp_{n-1} + 2)/6) - 1$$

This is because dp_n holds the answer for at most 2 children and the other section accounts for the rest. We multiply by 2 to account for both edges directions, and subtract 1 because a single path is isomorphic.

This obviously doesn't handle all cases, but all other cases can be found in the following form. Let $t_{up,k}$ be a tree where the root has 2 children and the edges are directed up and the longest path is k, and let $t_{down,k}$ be a tree where the root has 2 children and the edges are directed down and the longest path is k. Then all other cases are $t_{up,k}$ which exists on some path of length l to and connects to $t_{down,n-k-l}$

We can count every other case as $\sum_{i=0}^{n-1} (dp_i - 1) * dp2_{n-1-i}$

This works because we pretend the path is always length 1, then if we do $dp_i * dp2_{n-1-i}$ we handle all cases except for when the $t_{up,k}$ is empty, and that only happens one time.

Here is my submission: 117272002

→ Reply

MidoriFuse

5 years ago, hide # ^ | ☆ ← Rev. 2 0

Sorry I understand everything now D: took me too long

→ Reply

Huah

5 years ago, hide # ^ | ☆ ← Rev. 3 0

Good!

→ Reply

1443356159

5 years ago, hide # ^ | ☆ 0

We can count every other case as ∑n−1i=0(dpi−1)∗dp2n−1−i????

→ Reply

BASANT_KUMAR

5 years ago, hide # | ☆ ← Rev. 2 0

In problem B: my solution passed the pretests but giving time limit exceeded in test 4. It runs in O(nlogn) time I. Here is my submission Can someone help me finding what's wrong in my code?

→ Reply

lbw

5 years ago, hide # ^ | ☆ 0

In your code's solve, second range must be time limit exceeded.

the range is $O(n)$.And the min_element also is $O(n)$.so it's $O(n^2)$.

→ Reply

ethicalhacker365

5 years ago, hide # | ☆ 0

can anybody please tell in B why its optimal to always choose negative elements

→ Reply

viyaaat

5 years ago, hide # ^ | ☆ 0

the time complexity of min_element() is O(n), you are doing it (n-1) times in the worst case. you should think of propagating from the start only.

→ Reply

viyaaat

5 years ago, hide # ^ | ☆ 0

because their maximum element is negative and absolute of all differences is greater than or equals to zero that is greater than negative maximum element.

→ Reply

yaoveil

5 years ago, hide # ^ | ☆ ← Rev. 4 0

Suppose we choose more than 1 positive numbers a, b..., |a — b| < max(a, b), that means we can't choose more 1 positive numbers. Suppose all the numbers we choose is non-positive numbers, |ai — aj| >= max(ai, aj) is always true. So we just firstly choose all the non-positive numbers from the array, then calculate the minimum distance between these chosen numbers, we call it mn. And then check the original array if there is a positive number that is no greater than mn.

→ Reply

dark___horse

5 years ago, hide # | ☆ +6

Problem B Div-2 — It's easy to prove that a strange subsequence can't contain more than one positive element.

Can someone please prove this ?

→ Reply

berserker02

5 years ago, hide # ^ | ☆ +8

Assume that it contains two positive elements — say 2 and 5 . Max is 5. Difference is 3 . Similarly, it is easy to see that difference of two positive numbers will always be less than the greater element of those two numbers. Hence taking more than one positive number is not possible

→ Reply

beetroot

5 years ago, hide # ^ | ☆ 0

For two positive elements, we have the property that their absolute difference will be strictly smaller than the larger element. So, you can clearly see that this violates the strange condition

→ Reply

Stream_Cipher

5 years ago, hide # | ☆ ← Rev. 3 0

can anyone find error on my code for Div2 C failing on test case 2. I can't find any error ,i checked it more than 10 times ,any help will be really appreciated

→ Reply

toph3110

5 years ago, hide # | ☆ +8

In div1 C / div2 E , Can someone explain the part 3 , i.e. "*Among the vertices inside it, find the maximum number of vertices such that none of them is the ancestor of the other in Keshi's tree.*" . How are we doing it .

→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 2 +21

izlyforever

When you give a dfs order of a tree, there will be **no pair** satisfies

$in[u] < in[v], out[u] < out[v]$

and u is a ancestor of v if and only if $in[u] < in[v] \leq out[v] \leq out[u]$. so,

if neither of u or v is the ancestor of the other, then $[in[u], out[u]), [in[v], out[v])$ will be disjoint interval. and `std::set` is enough to maintain it.
→ Reply

5 years ago, hide # ^ | ☆ +26

shiftyblock

which also means euler tour generates a set of laminar ranges

-**PurpleCrayon**
→ Reply

5 years ago, hide # ^ | ☆ 0

klexis

Sounds like something important. What does it mean tho'?
→ Reply

5 years ago, hide # ^ | ☆ +5

shiftyblock

all ranges of tin and tout are either completely disjoint or contained within each other
→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 2 +8

Bishop

So, when you are at a leaf you have a set of intervals with you corresponding to the path from 1 to the that leaf. And you want to know maximum number of non-overlapping segments? Could you explain how you implemented it using `std::set` ?

UPD: Never mind. I got the idea. We can maintain only the disjoint intervals
→ Reply

5 years ago, hide # ^ | ☆ 0

klexis

Do you mean to say that in that set we only keep those intervals which will guarantee the max ans? If you do then won't it cause a problem while erasing a certain node? If not then what do you mean?
→ Reply

5 years ago, hide # ^ | ☆ +3

Bishop

Yes, we are maintaining the intervals that would cause maximum answer... The idea is that whenever you add a new segment in the set, then there are only three cases —
(1) The new segment doesn't intersect with any (in this case we simply add it)
(2) It comes inside some existing segment (so we remove the existing segment and add this one)
(3) If this segment covers some additional segments within it (we don't add it)..

So, whenever we add a segment it removes $\mathbf{atmost\ 1}$ segment from the set. We can keep track of which segment removes which one, and any change can be rolled back easily.
→ Reply

klexis

5 years ago, hide # ^ | ☆ +3

Oh that's brilliant. Thanks

→ Reply

EAT
SLEEP
<CODE/>
REPEAT

put_peace

5 years ago, hide # | ☆ +7

Bye Bye **Specialist**!!
Hello **Pupil**!!

→ Reply

parveen1981

5 years ago, hide # | ☆ +1

In Div 2 C, I think the condition should be $a_u \geq a_v$ for the first case and similarly $a_u \leq a_v$ for the second case. **AliShahali1382**

→ Reply

Tet

5 years ago, hide # ^ | ☆ 0

No, the case $a_u = a_v$ is handled in the third (otherwise) part.

→ Reply

parveen1981

5 years ago, hide # ^ | ☆ 0

No it's not handled in the third part. Because beauty of tree will still depend on a_v.

→ Reply

Tet

5 years ago, hide # ^ | ☆ 0

Ah, I see what you mean. I should rephrase that part.

It's better to compare the number of vertices p such that $a_p > a_v$ and the number of vertices q such that $a_q < a_v$. Thanks for noticing!

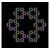
→ Reply

kj1809

5 years ago, hide # | ☆ +12

Very good round. ! Thanks for this round ! Graph and DP is always fun to solve in a contest. !

→ Reply



paras

5 years ago, hide # | ☆ +11

In Div-1 C [Editorial]: What are efficient ways to compute below?

Among the vertices inside it, find the maximum number of vertices such that none of them is the ancestor of the other in Keshi's tree.

→ Reply

Rick_S

5 years ago, hide # | ☆ +7

This one of the best contest I have ever given on codeforces.

→ Reply

God_knows___

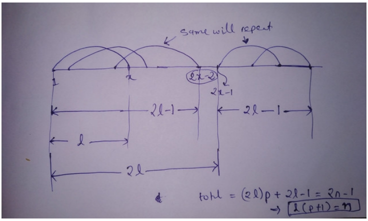
5 years ago, hide # | ☆ ← Rev. 3 +4

Can anyone explan in 2D,when $x \leq n$,why each length must be a divisor of n?

→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 3 +5

put_peace



Here, 1 pairs up with x and $x <= n$. Let say pattern repeats $p+1$ times, the bottom right equation in the image is for total length which is $2n-1$. (I have numbered from 1 to $2n$).
$l(p+1) = n$ tells us that l must be divisor of n.
→ Reply

God_knows___

5 years ago, hide # ^ | ☆ +1

thans a lot!
→ Reply

N-coder

5 years ago, hide # | ☆ ← Rev. 2 0

Did anyone try solving Div2-D by solving the complementary problem??

The complementary problem was to find the total number of bad pairings, where a pairing is considered to be bad if, for at least two segments A and B among those, the following condition holds:
$1)$ A and B don't lie completely inside each other.
$2)$ A and B don't have the same length.

I was able to find exactly two segments A and B, which will follow the above condition(if I am not wrong in computing that xD), but could not reduce it further. Is it even possible this way or is there any other approach to solve it??
→ Reply

jd_299

5 years ago, hide # | ☆ +6

I referred the editorial for Div2(C)/Div1(A)... But my solution failed after multiple attempts. Can someone please help me out? My solution
117289608
→ Reply

Tet

5 years ago, hide # ^ | ☆ +6

You forgot to call dfs.

Adding `dfs(1 , 0)` should fix your code...
→ Reply

N-coder

5 years ago, hide # ^ | ☆ ← Rev. 2 +7

Clear the global arrays before every test case as well, apart from calling the dfs function.
→ Reply

Tet

5 years ago, hide # ^ | ☆ +10

Yeah ... also his maxn had an extra 0.

Fixed code
→ Reply

5 years ago, hide # | ☆ +4

ShubhamAvasthi

"To be continued ..." in Div. 1 C's editorial?
→ Reply

5 years ago, hide # ^ | ☆ -7

ShubhamAvasthi

At the time of posting, the editorial was incomplete and said "To be continued ...", now it has been fixed.

I really don't deserve the downvotes.
→ Reply

5 years ago, hide # | ☆ 0

Hamim99

dpforces
→ Reply

5 years ago, hide # | ☆ +3

0rez

Can someone describe the data strcuture used for Div1 C? It just displays "to be continued".
→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 4 +16

yet_another_alt_account

I used a segment tree.

`tin[u]` is the entry time of the vertex u and `tout[u]` is the exit time of the vertex u during dfs.

1) To insert a node: First check if any of its ancestor is already inserted. If yes, remove the ancestor. Now, to insert the node, update the value in segment tree at the index `tin[u]` as `tout[u]`.

2) To delete a node: Update the value in segment tree at the index `tin[u]` as `0`.

3) To check if any ancestor is present: If current node is u, any ancestor v will satisfy the property `tin[v] < tin[u] && tout[v] > tout[u]`, so to check if any ancestor is already inserted, the range max query on this segment tree will be `max_query(1, tin[u] - 1) > tout[u]` if an ancestor in inserted.

For reference you can look at 117219173.
→ Reply

5 years ago, hide # ^ | ☆ 0

0rez

Thanks
→ Reply

5 years ago, hide # ^ | ☆ 0

BeruboIV

Won't there be a situation when it is better to remove the current node instead of the ancestor?
→ Reply

5 years ago, hide # ^ | ☆ +3

yet_another_alt_account

Nope, all the subsets which are satisfied by any ancestor will also be satisfied by this vertex, but not vice-versa.
→ Reply

5 years ago, hide # ^ | ☆ 0

BeruboIV

Wait, so if the current node is itself an ancestor of some of the previous nodes in the path, then does that mean it is the current node that will be removed?

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

yet_another_alt_account

"It's easy to see that the vertices in the data structure always form a path from root to some vertex v in Soroush's tree." so v (the current node) will never be an ancestor of any node currently in the data structure.

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

BeruboIV

No, I meant what happens if the current node (v) is an ancestor of some node in the data structure according to Keshi's tree?

→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 3 ▲ +13 ▼

Tet

Then we just ignore it and won't insert it.

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

BeruboIV

Thanks, I will try to implement it with this thing in mind.

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

PurpleThinker

A bit of a late reply, I am currently trying to wrap my head around this segtree solution implementation, but I have a question: how does the code actually take into consideration the case when `u` is an ancestor of a previously visited node `v` (by "ancestor" I mean an ancestor in Keshi's tree)? What I mean is that, in the DFS function in Soroush's tree, how does the code take into consideration the case when the current node `u` is an ancestor (in Keshi's tree) of a previously visited node `v` by the DFS?

→ Reply

5 years ago, hide # | ☆ ▲ 0 ▼

beetroot

In div2 D, lets say n=4(8points) And lets say we pair 1 with 4 What are the rest of the pairs? Can anyone list them?

→ Reply

5 years ago, hide # ^ | ☆ ← Rev. 2 ▲ 0 ▼

weakestOsuPlayer_244

You can't pair 1 with 4 that's invalid. If u draw on paper u can see why. You can pair 1 with 3 and 2 with 4 followed by 5 with 7 and 6 with 8.

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

beetroot

But when we calculate the divisors of n, we get 1,2 and 4, right?

→ Reply

5 years ago, hide # ^ | ☆ ▲ 0 ▼

weakestOsuPlayer_244

You don't include n amongst divisors. (I think maybe this wasn't mentioned in editorial)

for n = 4 these are the valid equal length pairings.

1. (1,2) (3,4) (5,6) (7,8) for divisor = 1
2. (1,3) (2,4) (5,7) (6,8) for divisor = 2

which is 3 — 1 = 2.

→ Reply

beetroot
5 years ago, hide # ^ | ☆ ← Rev. 3 0

resolved

→ Reply

weakestOsuPlayer_244
5 years ago, hide # ^ | ☆ ← Rev. 2 0

One minute it is correct my bad.. actually the way I have implemented is a bit different

there is one more pair (1,5) (2,6) (3,7) (4,8)

You may want to edit your comment. I am sorry for the mistake.

→ Reply

weakestOsuPlayer_244
5 years ago, hide # ^ | ☆ 0

these are actually those equal length pairings which do not have anything lying inside them

→ Reply

beetroot
5 years ago, hide # ^ | ☆ 0

But this is when x>n In the other case, when x<=n, we still cannot take x=n right?

→ Reply

motorcycle
5 years ago, hide # ^ | ☆ ← Rev. 4 0

According to the tutorial's method, author discuss the problem for 2 cases x <= n and x > n, and as we can see, in the former, we can only take length at most n − 1,also in the solution, d(n) is proper divisor of n. But from my point of view, talk in x <= n + 1 and x > n + 1 is more natural, from then on, length is <= n now, d(n) is all divisors of n, including itself, the formula has finally become $dp_n = \sum_{i=1}^{n-1} dp_i + d(n)$. also it is no need to initialize $dp_0 = 1$ (which has no useful purpose) anymore.

→ Reply

weakestOsuPlayer_244
5 years ago, hide # ^ | ☆ 0

yes we can't take when x = n.

the one case which is contributing to total number of divisors is from the x > n case and nothing lying within it.

If you see my code I have added + 1 separately in dp[i] = dp[i-1] + 1 + count[even divisors of 2*n]

and from the number of even divisors I have subtracted -1;

strangely the net sum ends up being dp[i] = dp[i-1] + divisors.

even divisors of 2 * n is same as odd divisors of n

→ Reply

me3tazzzz
5 years ago, hide # | ☆ 0

I really loved the problems!

→ Reply

Ordinary_Oier
5 years ago, hide # | ☆ 0

Sorry to correct one possible little mistakes about the editorial of problem 1528A. In case 3(p=q),to maximize the beauty of the tree, au must be rv or uv.For example, if au=3,lv=0,rv=7, p=q=4, av=4,5,9,5,1,2,0,-1, the original beauty is 21.If change au into rv, the beauty will be 35.(In a word,I mean " The beauty of the tree doesn't depend on av" is not very rigorous.) However,thanks for your editorials again! They have helped me a lot!

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

Fixed, thanks for noticing.

→ Reply

BlueTulpis
5 years ago, hide # | ☆ +11

Can someone tell, how to solve Div2C/Div1A if we need to minimise the sum instead of maximizing?

→ Reply

Tet
5 years ago, hide # ^ | ☆ +10

Just a thought: We can define $dp_{v,j}$ as the minimum beauty of v's subtree if a_v is equal to j. This is too slow though, because I don't see any fact like the one in the maximizing version in here :(

→ Reply

BlueTulpis
5 years ago, hide # ^ | ☆ 0

I think that would work, if we have small ranges, but how to do if we have large ranges as in the problem?

→ Reply

Tet
5 years ago, hide # ^ | ☆ +14

That's why I said it's too slow. Maybe we can prove that $dp_{v,j}$ is convex or maybe not ...

But it should be a fun problem if it turns out to be convex :D

→ Reply

Laplaceisin
5 years ago, hide # | ☆ 0

how would we solve the div2 problem A if we were asked to find the max no of operations instead of max no of elements to be deleted?

→ Reply

Tet
5 years ago, hide # ^ | ☆ ← Rev. 3 +10

The algorithm I provided deletes 1 element at a time, thus does the maximum number of operations. It's worth noting that the answer to both of the problems is the same.

→ Reply

nischay193
5 years ago, hide # | ☆ 0

Getting TLE on test case 4 of Div2 C. It follows the editorial explanation. Any help would be appreciated. Here's my submission.

→ Reply

Tet
5 years ago, hide # ^ | ☆ +10

Add `cin.tie(0) , ios::sync_with_stdio(0);` to your main function and it should be fine.

→ Reply

nischay193
5 years ago, hide # ^ | ☆ 0

Thanks a lot. It works now. But why does it not work within the time constraints? Most of the n <= 1e5 cases work fine without fast I/O in O(nlogn) sorting time. Why does this solution, which is supposed to be O(n), fail to run in the required time?

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

It's not as simple as that, the sum of n over all test cases can be as large as $2 \cdot 10^5$, there can be approximately $2 \cdot 10^5 \cdot (5 + 9 + 9)$ characters in the input which is quite a lot.

→ Reply

nischay193
5 years ago, hide # ^ | ☆ 0

I'm sorry I did not understand, could you elaborate?

→ Reply

Tet
5 years ago, hide # ^ | ☆ +8

I said that you might need to read more than 4600000 characters from the input (4.5e6). cin itself is slow so you can't read that many characters in time. So you have to add that magical line to your code to make it work faster.

This is because by default, before you call the function `cin`, the call `cout << flush` is done. Adding `cin.tie(0)` unties `cin` from `cout` and stops this.

→ Reply

Underdog_123
5 years ago, hide # | ☆ ← Rev. 2 0

Can anybody please help me what is wrong with this code it is showing TLE test set 4 of question 3 Please help Thank you in advance

Here is my code:
https://codeforces.com/contest/1528/submission/117327246

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

You forgot to add `cin.tie(0) , ios::sync_with_stdio(0);` to your main function.

→ Reply

Elder_Giang
5 years ago, hide # | ☆ ← Rev. 3 0

First time to solve C without fst though I used O(n(logn)^2) algorithm.

→ Reply

antonykelvin7
5 years ago, hide # | ☆ 0

Can someone explain to me for 1528A — Parsa's Humongous Tree why every vertex needs to be either lu or ru. I'm finding it difficult to understand. Thanks

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

It doesn't, but we proved that an optimal assignment exists that each a_v is either l_v or r_v.

→ Reply

HasNyen9
5 years ago, hide # | ☆ 0

what does it mean by ans += a[i] != mn; in the 2nd for loop?

→ Reply

Tet
5 years ago, hide # ^ | ☆ +10

If $a_i \neq mn$: $ans += 1$

→ Reply

antonykelvin7
5 years ago, hide # | ☆ 0

Can someone tell me why this code times out? (https://codeforces.com/contest/1529/submission/117339337)

The problem is 1528A — Parsa's Humongous Tree. Thanks!

→ Reply

Tet
5 years ago, hide # ^ | ☆ +10

Please read the comments above ... at least two people have already asked this.

You forgot to use `ios::sync_with_stdio(0), cin.tie(0);`.

→ Reply

rehan_amaan_how_so_good
5 years ago, hide # | ☆ +10

In problem Div1C:

> *"Now to check whether any vertex u in the subtree of vertex v exists in S,..."*

You don't need this check at all to AC. This is because of the condition that $p[i] < i$ for all i in both trees. Assume that x is a parent of y in any tree, then that ensures that $x < y$. You can see that it can't be true that x is a parent of y in Soroush's tree (and is therefore in the stack of nodes beforehand when we start DFS at y), and y is a parent of x in the other tree, since that would require $y < x$ and $x < y$.

→ Reply

itti_da
5 years ago, hide # | ☆ 0

In 1528A — Parsa's Humongous Tree, i used a bit different approach in dp which seems quite similar to me with the one provided in editorial . But it is not passing the pretests and i can't figure out what is wrong with my approach. I am new to dp. Can someone help me out?
Code using my own approach: my approach Code using editorial's approach: editorial approach

→ Reply

Polar_
5 years ago, hide # | ☆ ← Rev. 3 +8

In problem C official solution, because of the way the trees are given in the input (each vertex in both trees has higher label than its ancestors), there's actually no need to check the case where $res = -2$. For any pair of vertices u, v, if u is ancestor of v in the first tree $(u < v)$, v can't be ancestor of u in the second tree as it would imply $u > v$. So, you always insert the current vertex, you just need to check if there's any ancestor of it and if there is, just remove it :)

Here's the submission 117369367 with the corresponding assert.

→ Reply

Selva_1
5 years ago, hide # | ☆ ← Rev. 4 0

In the editorial solution of 1528A, the proof claims we can get a better result by changing av to lv or rv because the contribution from its children increase in this process. But at the same time it can reduce the contribution from its parent right? How do we say that the net result of changing av is an increase in beauty?

→ Reply

Tet
5 years ago, hide # ^ | ☆ +18

> *In the editorial solution of 1528A, the proof claims we can get a better result by changing av to lv or rv because the contribution from its children increase in this process.*

No, not v's children but the vertices adjacent to v, this includes v's parent as well.

→ Reply

Selva_1
5 years ago, hide # ^ | ☆ 0

Got it!! Thanks

→ Reply

Ashish.iitp
5 years ago, hide # | ☆ ← Rev. 2 0

https://codeforces.com/contest/1529/submission/117382997 This is my solution for problem C. It is showing TLE. Can anyone debug my solution,Pease.

→ Reply

Tet
5 years ago, hide # ^ | ☆ +18

There's nothing wrong with your code. The input is quite large so you have to add `cin.tie(0) , ios::sync_with_stdio(0);` before reading input to do it faster.

→ Reply

Little-aztl
5 years ago, hide # | ☆ 0

Well, the editorial of the 1528C - Trees of Tranquility has some mistakes.

In the sixth to last line, $p \geq st_v 0$ should be changed to $p \geq st_v$.

"is less than st_v" should be changed to "is less than ft_v".

→ Reply

jalanprakhar
5 years ago, hide # | ☆ 0

Q2 could've been solved using binary search as well which would reduce the time complexity. https://del.dog/binarysearchsolution.txt here is my solution using binary seach.

→ Reply

Dardy
5 years ago, hide # | ☆ 0

Here is a stupid solution for Div.1 C:

The problem is as such: choose a maximum subset of a path in Soroush's tree such that if u and v are in that subset neither u nor v is an ancestor of the other in Keshi's tree.

The problem can be solved as such: dfs on Soroush's tree, take node u if none of its subtree in Keshi's tree is taken. If u has ancestor p in Keshi's tree that was already taken, greedily take u in place of v, this works because a lower node is an ancestor of fewer nodes, in another word: picking u instead of p offers the chance to pick v where v is a child of p and a sibling of u.

To solve this we'll need two kinds of queries: 1. on a path from root to u find if a node is already taken, 2. in the subtree of u find if a node is already taken. Both queries can be done by using a an approach similar to here.

Similar to the editorial we store st and ft for each node, notice that for the range $[0, st_u]$ a node exists exactly once if it is on the path from root to u and may exist twice otherwise, thus we just need to focus on nodes that occurred a single time. To add a node we update the segment tree at st_u and ft_u to be u, to remove a node we set them to 0.

Since at most one ancestor is taken at a time, we can implement an xor segment tree on the dfs order and query the range $[0, st_u]$; undesirable nodes cancel themselves out, all other nodes will be 0 except for (potentially) one node: p which we will then remove and take u instead. To solve 2, I just implemented an or segment tree (max would work as well) to find if a node exists in subtree.

my solution, not pretty

→ Reply

ehnryx
5 years ago, hide # | ☆ ← Rev. 3 0

For problem F, how do you go from the sum with binomial coefficients to the sum with Stirling numbers? Also, should the sum be from CNT=1 to n?

→ Reply

ehnryx
5 years ago, hide # ^ | ☆ ← Rev. 3 0

I got it.

We can interpret $\sum_{r=1}^{n} \binom{n}{r} \cdot n^{n-r} \cdot r^k$ as the number of functional graphs whose vertex set is $V = A \cup B \cup Z$ such that

1. $|A| = n$, $|B| = k$, and $Z = \{z\}$ with $f(z) = z$,
2. $f(V) \subset A \cup Z$ (no element of B has any incoming edges),
3. Let $C = \{a \in A : f(a) = z\}$ be the elements in A that map to z. Then, $|C| = r$ and $f(A \setminus C) \subset A$ (the elements in A that don't map to z all map to some element in A),
4. $f(B) \subset C$.

This interpretation is correct because there are $\binom{n}{r}$ ways to choose the subset C, then n^{n-r} ways to choose how $A \setminus C$ maps to A, and r^k ways to choose how B maps to C.

We can count these functional graphs in another way. First, let $i = |f(B)|$.

1. Choose how to map B to $f(B)$ by first partitioning B into i sets. There are $\left\{ {k \atop i} \right\}$ ways to partition B into i sets and $i!$ ways to order these sets.
2. There are $\binom{n}{i}$ ways to choose a subset of A to be $f(B)$.
3. There are $(n+1)^{n-i}$ ways to choose how $A \setminus f(B)$ maps to $A \cup Z$.

→ Reply

callme_oj
5 years ago, hide # | ☆ 0

Can anyone explain why pairing is unique for x <= n in Div2 D/ Div1 B.

→ Reply

ch0pikus
5 years ago, hide # | ☆ 0

There's an error in tutorial of Div1B.

$dp[i]$ is not equal to $\sum_0^{n-1} dp_i + D(n)$, but equals to $\sum_0^{n-1} dp_i + D(n) - 1$ instead.

That's because length must be a divisor of N **and** less than N, otherwise $x > N$.

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

Oops! Sorry, it's fixed now; Although it might take some time for the changes to appear. Thanks for noticing!

→ Reply

Solnechniy_kray
5 years ago, hide # | ☆ 0

Can somebody explain magical number 63 in div.1-D solution? From the first glance it implies that answers can't exceed 63...

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

What are you talking about?

→ Reply

Solnechniy_kray
5 years ago, hide # ^ | ☆ 0

1528D -> official solution (not editorial but code)

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

I'm assuming that you are talking about `memset(a, 63, sizeof a)`.

Memset works by writing values byte by byte based on the given value. So if you want to set an array to large values, use memset with 63, this will set all values to 1,061,109,567.

→ Reply

Solnechniy_kray
5 years ago, hide # ^ | ☆ 0

Got it! Thnx for explanation.

→ Reply

augsaksham
5 years ago, hide # | ☆ 0

B. Sifid and Strange Subsequences

Considering The Test Case

14 9276 -5771 -783 -6209 2041 -9286 -8065 3975 -2961 2689 -1680 9924 1483 3067

The -ve numbers are -9286,-8065,-6209,-5771,-2961,-1680,-783

And the minimum +ve number is 1483

I am confused about how can just adjacent checking ensure that the difference of pairs in the subarray satisfy the condition of strange sequence. For example abs((-9286-(-8065)) is not > 1483 similarly it holds true for -1680 and -783. Can someone please explain me why is this intuition incorrect .

→ Reply

Tet
5 years ago, hide # ^ | ☆ 0

Consider this task: Given a non-decreasing array a, you must find a pair of elements a_i and a_j such that $i < j$ and $|a_i - a_j|$ is minimum. Obviously, there exists an answer such that $j - i = 1$, because otherwise we could have chosen a_{i+1} without getting a worse result.

→ Reply

tgp07
4 years ago, hide # | ☆ ← Rev. 2 0

Deleted

→ Reply

4 years ago, hide # ^ | ☆ 0

It TLEs because you are an illiterate moron who refuses to read the other comments

Tet

other comments.

It's been asked here, here, here and here.

Also your fixed code: 146310603

→ Reply

tgp07

4 years ago, hide # ^ | ☆ ▲ +5 ▼

My fault, I read the other comments, but forgot to delete mine,

→ Reply

Codeforces (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform
Server time: Jul/27/2026 08:06:00$^{\text{UTC-4}}$ (n2).
Mobile version, switch to desktop version.
Privacy Policy | Terms and Conditions

Supported by